

PROJECT DOCUMENTATION

A Comprehensive Measure of Well-Being

Submitted in Partial Fulfillment of the Requirements for the SmartBridge AI & Machine Learning Internship Program

Developed By

Team ID: _____

Team Members:

- Santhosh Masupalli
- Polisetty Venkata Jagadeesh
- Mukiri Praveen Ram
- Varshit Ventrapragada

**Guided By
SmartBridge**

**Submitted On
4 th July 2026**

Table of Contents

Section	Heading
1	Project Description
2	Technical Architecture
3	Installation steps, usage Instructions
4	Project Workflow Overview
5	Frontend Development
6	Deployment & Testing
7	Home Page (index.html)
8	Prediction Result Page (result.html)
9	Testing Summary
10	Conclusion
11	Future Enhancements
12	References
13	Project Summary
14	links

Abstract

A Comprehensive Measure of Well-Being is a Machine Learning-based web application developed to predict the **Human Development Index (HDI)** using key socio-economic indicators. The system analyzes **Life Expectancy at Birth, Expected Years of Schooling, Mean Years of Schooling, and Gross National Income (GNI) per Capita** to estimate the HDI score accurately. A **Linear Regression** model is trained using historical HDI data and integrated into a Flask web application. Users can enter the required input values through a simple web interface and instantly receive the predicted HDI score along with its corresponding development category (Very High, High, Medium, or Low Human Development). This project demonstrates the practical application of Machine Learning in analyzing development indicators and supports students, researchers, and policymakers in understanding and evaluating human development efficiently.

1. Project Description

A Comprehensive Measure of Well-Being is a Machine Learning-based web application developed to predict the Human Development Index (HDI) of a country using key socio-economic indicators. The application analyzes Life Expectancy at Birth, Expected Years of Schooling, Mean Years of Schooling, and Gross National Income (GNI) per Capita to estimate the Human Development Index accurately.

The system is built using Python, Flask, Scikit-learn, Pandas, and NumPy. A Linear Regression model is trained using the HDI dataset and saved as a serialized model (HDI.pkl). Users interact with the application through a simple web interface where they enter the required indicators and instantly receive the predicted HDI score along with its corresponding development category (Very High, High, Medium, or Low Human Development).

The application demonstrates how Machine Learning can simplify the analysis of socio-economic indicators and assist students, researchers, policymakers, and development organizations in understanding and evaluating human development efficiently.

| Usage Scenarios

➤ Scenario 1 — Policy Analysis

A policymaker wants to estimate the Human Development Index of a country after updating the values of life expectancy, education, and income. The application predicts the HDI score instantly and classifies it into the appropriate development category.

➤ Scenario 2 — Comparative Research

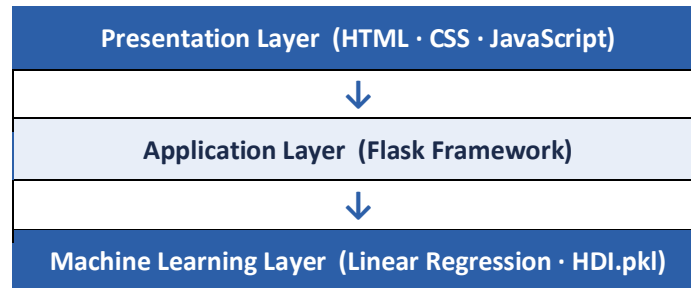
A researcher compares the development status of multiple countries by entering different socio-economic indicators. The system predicts HDI values quickly, making comparative analysis easier and more efficient.

➤ Scenario 3 — Learning Tool

A student learning about the Human Development Index uses the web application to understand how changes in education, income, and life expectancy affect the predicted HDI value and overall development category.

2. Technical Architecture

The proposed system follows a three-layer architecture consisting of the Presentation Layer, Application Layer, and Machine Learning Layer



| Layer Description

- **Presentation Layer**

Provides a user-friendly web interface developed using HTML, CSS, and JavaScript. Users enter Life Expectancy, Expected Years of Schooling, Mean Years of Schooling, and Gross National Income (GNI) per Capita.

- **Application Layer**

Developed using the Flask framework. It receives user inputs, validates the data, loads the trained Machine Learning model (HDI.pkl), and processes prediction requests.

- **Machine Learning Layer**

Contains the Linear Regression model trained using the HDI dataset. Based on the supplied input values, the model predicts the Human Development Index and returns the predicted score along with its corresponding development category.

This architecture provides accurate prediction, modular design, easy maintenance, and efficient execution. Since this project uses a CSV dataset rather than a relational database, an ER Diagram is not required.

| Pre-requisites

- Python Programming
- Flask Framework
- Machine Learning using Scikit-learn
- Pandas and NumPy Libraries
- HTML, CSS, and JavaScript
- Jupyter Notebook
- Visual Studio Code

- Git and GitHub
- Basic Knowledge of Linear Regression
- CSV Dataset Handling

Installation Steps

➤ Hardware Requirements

- Intel Core i3/i5 or higher
- 4 GB RAM (8 GB recommended)
- 500 MB free disk space

Software Requirements

- Python 3.10 or above
- Visual Studio Code
- Jupyter Notebook
- Git (optional)

➤ Steps

1. Download or clone the project repository.
 - a. Open the project folder in Visual Studio Code.
 - b. Create a virtual environment (optional):
2. `python -m venv venv`
 - a. Activate the virtual environment.
 - i. Windows:
3. `venv\Scripts\activate`
 - a. Install the required packages:
4. `pip install -r requirements.txt`
 - a. Ensure the following files are present:
 - i. `app.py`
 - ii. `HDI.pkl`
 - iii. `requirements.txt`
 - iv. `templates/index.html`
 - v. `templates/result.html`
 - vi. `static/`
 - b. Start the Flask application:
5. `python app.py`
 - a. Open a web browser and visit:
6. `http://127.0.0.1:5000/`

➤ Usage Instructions

1. Launch the Flask application by running:

0. python app.py
2. Open the application in your web browser.
3. Enter the required input values:
 - Life Expectancy at Birth
 - Expected Years of Schooling
 - Mean Years of Schooling
 - Gross National Income (GNI) per Capita
4. Click the **Predict** button.
5. The application processes the input using the trained Linear Regression model.
6. The predicted **Human Development Index (HDI)** score is displayed.
7. The application also displays the corresponding **HDI category**:
 - Very High Human Development
 - High Human Development
 - Medium Human Development
 - Low Human Development

3.Project Workflow Overview

The project was executed across six milestones, progressing from model selection through to deployment and evaluation.

Milestone	Focus Area
1	Model Selection and Architecture
2	Core Functionalities Development
3	App.py Development
4	Frontend Development
5	Deployment & Testing
6	Conclusion & Future Enhancements

Milestone 1 — Model Selection and Architecture

1.1 Problem Identification

The Human Development Index (HDI) is one of the most widely used indicators for measuring the overall development of a country. It combines three important dimensions of human development: health, education, and income. Traditionally, estimating HDI requires collecting and analyzing multiple socio-economic indicators, which is time-consuming and requires domain expertise.

The objective of this project is to develop a Machine Learning-based web application that predicts the Human Development Index using four key indicators:

- Life Expectancy at Birth
- Expected Years of Schooling
- Mean Years of Schooling
- Gross National Income (GNI) per Capita

The application provides users with an easy-to-use interface where these values can be entered, and the trained model predicts the HDI score along with the corresponding development category.

- **Objectives**

1. Develop an intelligent HDI prediction system.
2. Train a Machine Learning model using HDI data.
3. Build a Flask web application for prediction.
4. Display the predicted HDI score and category.
5. Provide a simple and interactive user interface.

1.2 Dataset Collection and Analysis

The dataset used in this project contains Human Development Index indicators collected from publicly available Human Development datasets. It includes important socio-economic variables that influence the HDI of different countries.

- **Dataset Features**

- Country
- Life Expectancy at Birth
- Expected Years of Schooling
- Mean Years of Schooling
- Gross National Income (GNI) per Capita
- Human Development Index (Target Variable)

- **Data Preprocessing**

6. Imported the dataset using Pandas.
7. Checked for missing values.
8. Removed duplicate records.
9. Selected only the required features.

10. Converted numerical columns into appropriate data types.
11. Prepared the dataset for training.

Outcome: the processed dataset serves as the input for training the Linear Regression model, enabling accurate HDI prediction.

1.3 Machine Learning Model Selection

Several regression algorithms can be used for HDI prediction. For this project, Linear Regression was selected because the relationship between HDI and socio-economic indicators is continuous and numerical.

➤ Why Linear Regression?

- Simple and efficient
- Easy to interpret
- Suitable for continuous value prediction
- Fast training and prediction
- Good performance on structured datasets

➤ Libraries Used

- Pandas
- NumPy
- Scikit-learn
- Pickle
- Matplotlib
- Seaborn

1.4 Solution Architecture

The application follows a three-layer architecture, with a data layer supplying the source dataset.

Layer	Components
Presentation Layer	HTML, CSS, JavaScript
Application Layer	Flask Framework, Python
Machine Learning Layer	HDI.pkl Model, Prediction Engine
Data Layer	HDI.csv Dataset

The architecture allows users to submit development indicators through the web interface; the data is processed using the trained model, and the predicted HDI score is returned to the user.

1.5 Development Environment Setup

Component	Technology
Programming Language	Python 3.x
Framework	Flask
IDE	Visual Studio Code
Notebook	Jupyter Notebook
Version Control	Git & GitHub
Machine Learning	Scikit-learn
Data Processing	Pandas, NumPy
Visualization	Matplotlib, Seaborn
Frontend	HTML, CSS, JavaScript
Model Storage	Pickle (HDI.pkl)

The required Python libraries were installed using pip, and the project structure was organized into folders for the dataset, training notebook, Flask application, templates, static files, and the trained model.

Deliverables

- Configured project environment
- Installed all dependencies
- Created project folder structure
- Prepared the dataset
- Completed Machine Learning model setup
- Ready for Flask application development

Milestone 2 — Core Functionalities Development

2.1 Core HDI Prediction Features

The primary functionality of this project is to predict the Human Development Index (HDI) using a trained Machine Learning model. The application processes user-provided socio-economic indicators and generates the predicted HDI score along with the corresponding development category.

➤ Input Feature Collection

The application accepts the following user inputs:

- Life Expectancy at Birth
- Expected Years of Schooling
- Mean Years of Schooling
- Gross National Income (GNI) per Capita

➤ **Machine Learning Prediction**

12. Load the trained HDI.pkl model.
13. Convert user input into a Pandas DataFrame.
14. Predict the Human Development Index.
15. Display the predicted HDI value.

➤ **HDI Category Classification**

Based on the predicted score, the country is classified as:

- Very High Human Development
- High Human Development
- Medium Human Development
- Low Human Development

| 2.2 Flask Backend Implementation

➤ **Define Flask Routes**

- / → Home Page
- /predict → Predict HDI

➤ **Process User Input**

- Accept values submitted through the HTML form
- Validate numerical inputs
- Convert values into floating-point format

➤ **Model Integration**

- Load the trained Linear Regression model
- Generate the HDI prediction
- Send prediction results to the frontend.

Milestone 3 — App.py Development

Milestone 3 focuses on developing the core backend logic responsible for receiving user input, processing prediction requests, interacting with the Machine Learning model, and displaying the final prediction.

3.1 Main Application Logic

➤ Home Route

The home route loads the application's main webpage.

```
6
7  app = Flask(__name__)
8  model = pickle.load(open("HDI.pkl", "rb"))
9
```

➤ Prediction Route

The prediction route receives user input, prepares the input data, performs prediction using the trained model, and displays the result.

```
@app.route("/predict", methods=["POST"])
def predict():
    # 1. Read user input
    # 2. Convert values into floating-point numbers
    # 3. Create a DataFrame
    # 4. Load the trained model
    # 5. Predict HDI
    # 6. Determine HDI category
    # 7. Display the prediction result

    print(request.form)
    print("Predict route called")
    # Read values entered by the user
    life_expectancy = float(request.form["life_expectancy"])
    expected_schooling = float(request.form["expected_schooling"])
```

3.2 Preparing Input Data

The collected values are converted into a Pandas DataFrame with the same feature names used during model training.

```
# Create DataFrame with the same feature names used during
input_data = pd.DataFrame({
    "Life expectancy at birth": [life_expectancy],
    "Expected years of schooling": [expected_schooling],
    "Mean years of schooling": [mean_schooling],
    "Gross national income (GNI) per capita": [gni]
})
```

3.3 Generating the HDI Prediction

The trained Linear Regression model predicts the Human Development Index, and the result is rounded to three decimal places.

```
prediction = model.predict(input_data)
predicted_hdi = round(float(prediction[0][0]), 3)
```

3.4 Classifying the HDI

The application classifies the prediction into one of four categories:

```
if predicted_hdi >= 0.800:
    category = "Very High Human Development"

elif predicted_hdi >= 0.700:
    category = "High Human Development"

elif predicted_hdi >= 0.550:
    category = "Medium Human Development"

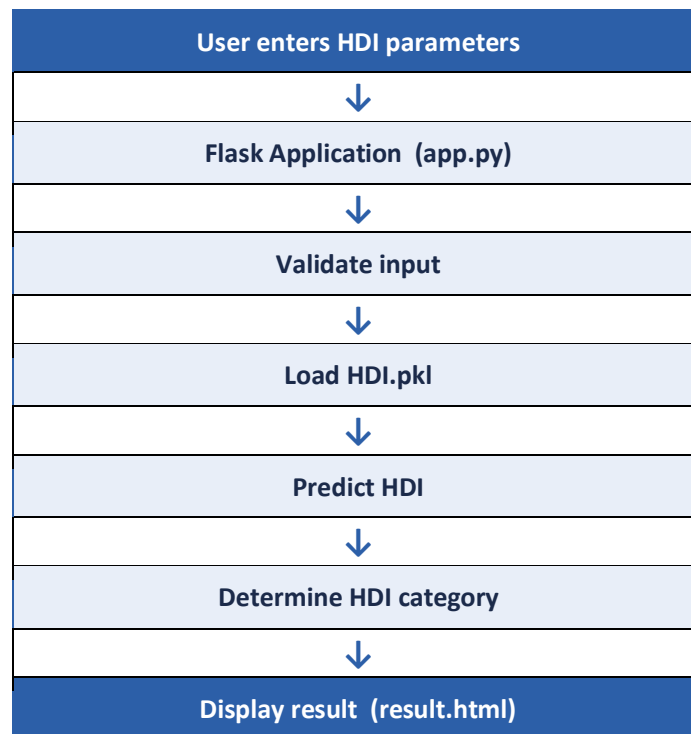
else:
    category = "Low Human Development"
```

3.5 Displaying the Prediction Result

After prediction, the application sends the predicted HDI score and category to result.html:

```
return render_template(
    "result.html",
    hdi_score=predicted_hdi,
    category=category
)
```

3.6 Overall Prediction Flow



Outcome: the backend successfully integrates the trained Machine Learning model with the Flask web application, enabling users to obtain accurate HDI predictions based on the provided socio-economic indicators.

Milestone 4 — Frontend Development

4.1 Designing the User Interface

The frontend of A Comprehensive Measure of Well-Being provides a simple and interactive interface that allows users to enter the required Human Development Index indicators and receive prediction results instantly.

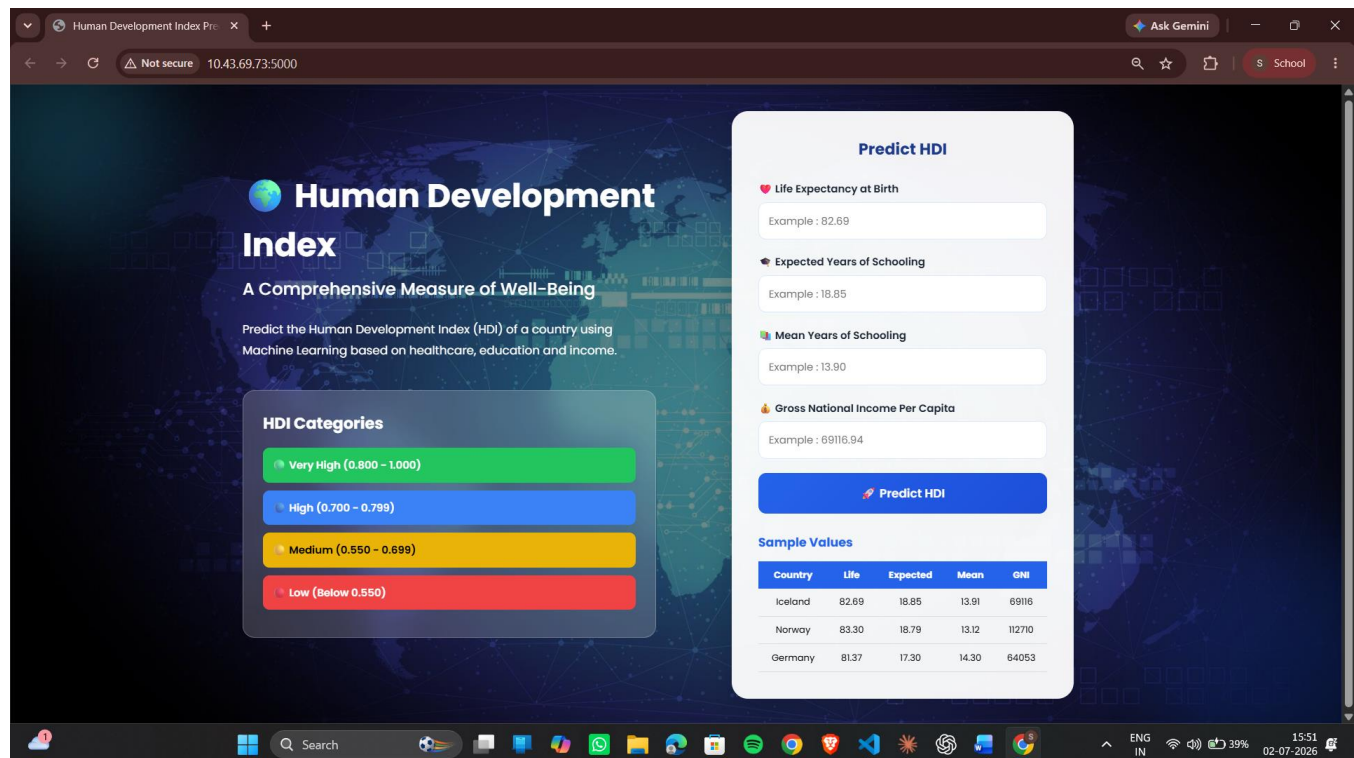
The interface was developed using HTML5, CSS3, JavaScript, and Flask Templates (Jinja2).

➤ Objectives

- Simple and attractive design
- Easy navigation
- Responsive layout
- User-friendly experience
- Fast interaction

➤ Input Fields

- Life Expectancy at Birth
- Expected Years of Schooling
- Mean Years of Schooling
- Gross National Income (GNI) per Capita



4.2 HTML & CSS Development

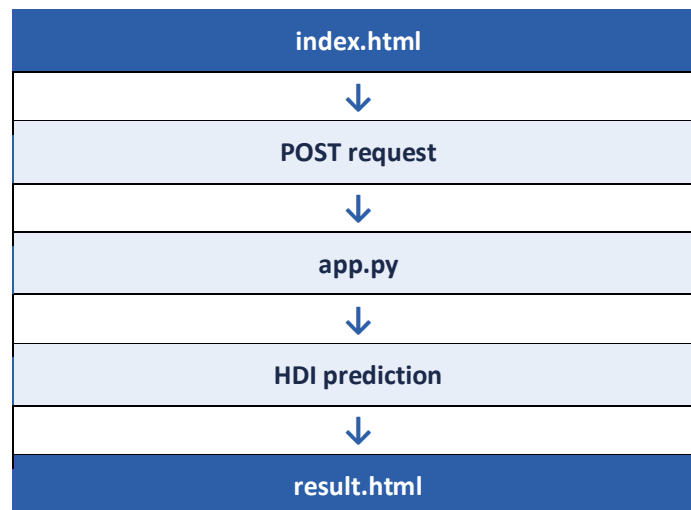
➤ HTML Features

- Responsive layout
- Input validation
- Predict button
- Clear labels
- Simple navigation

➤ CSS Styling

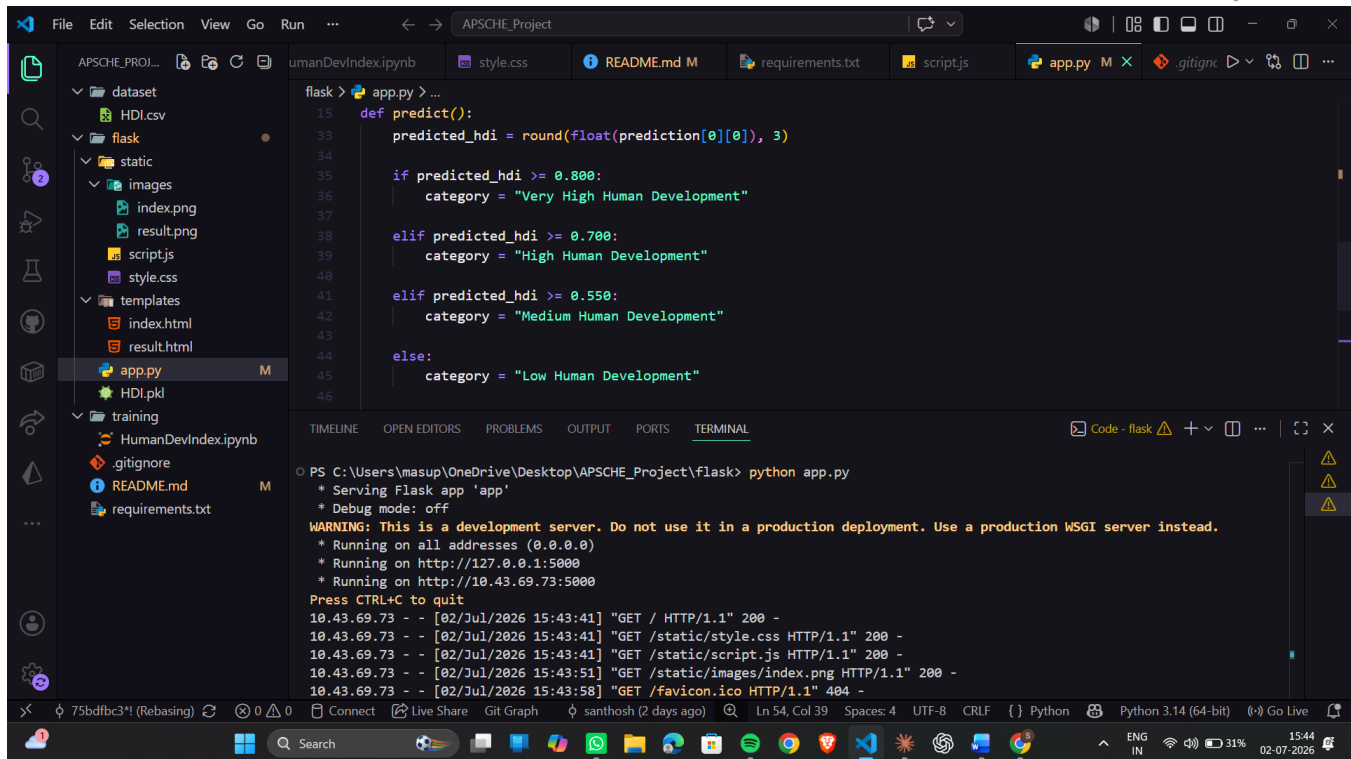
- Modern color scheme
- Responsive design
- Rounded buttons
- Input field styling
- Mobile-friendly layout

4.3 Flask Integration



➤ Backend Tasks

- Receive user input
- Validate data
- Load HDI.pkl
- Predict HDI
- Return prediction result



```

flask > app.py > ...
15 def predict():
33     predicted_hdi = round(float(prediction[0][0]), 3)
34
35     if predicted_hdi >= 0.800:
36         category = "Very High Human Development"
37
38     elif predicted_hdi >= 0.700:
39         category = "High Human Development"
40
41     elif predicted_hdi >= 0.550:
42         category = "Medium Human Development"
43
44     else:
45         category = "Low Human Development"
46
TIMELINE OPEN EDITORS PROBLEMS OUTPUT PORTS TERMINAL
o PS C:\Users\masup\OneDrive\Desktop\APSCHE_Project\flask> python app.py
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://10.43.69.73:5000
Press CTRL+C to quit
10.43.69.73 - - [02/Jul/2026 15:43:41] "GET / HTTP/1.1" 200 -
10.43.69.73 - - [02/Jul/2026 15:43:41] "GET /static/style.css HTTP/1.1" 200 -
10.43.69.73 - - [02/Jul/2026 15:43:41] "GET /static/script.js HTTP/1.1" 200 -
10.43.69.73 - - [02/Jul/2026 15:43:51] "GET /static/images/index.png HTTP/1.1" 200 -
10.43.69.73 - - [02/Jul/2026 15:43:58] "GET /favicon.ico HTTP/1.1" 404 -
  
```

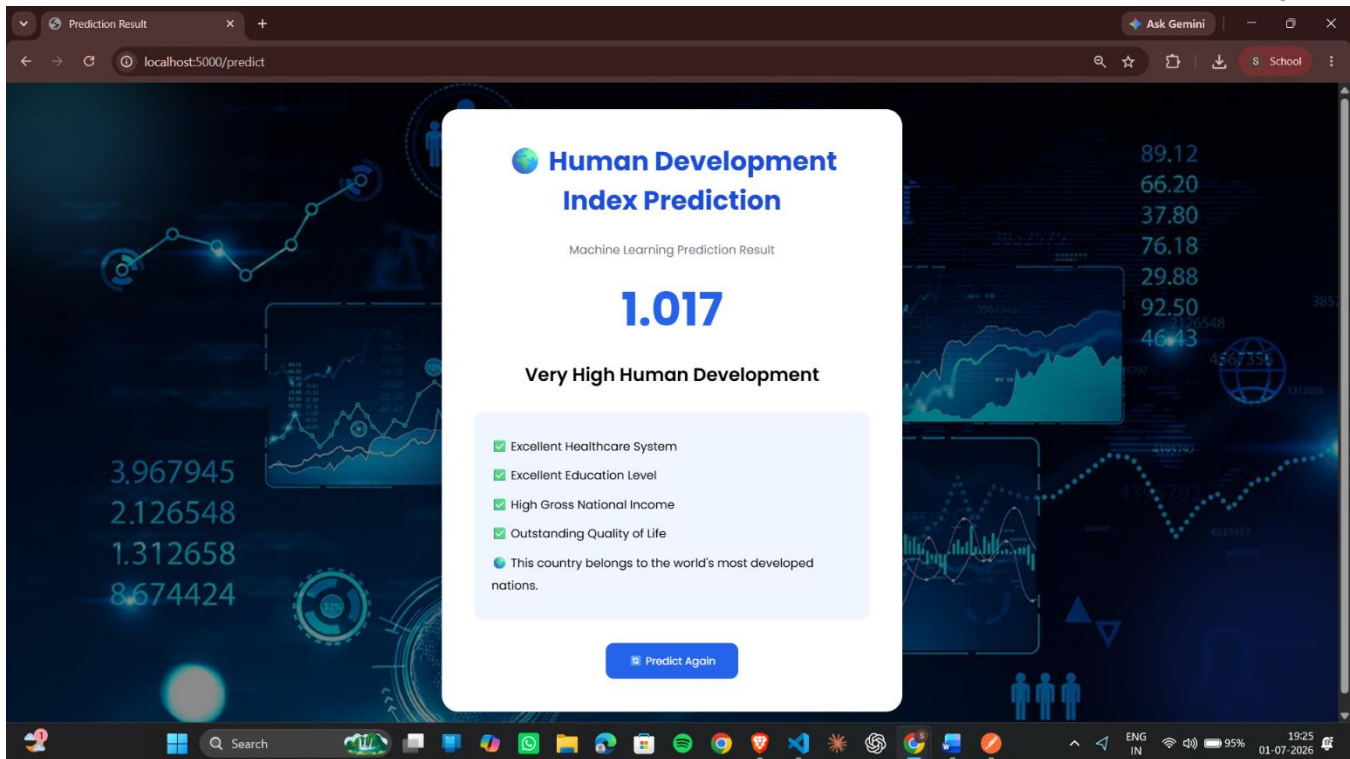
4.4 Result Page

The result.html page displays the predicted HDI score and the corresponding development category.

Field	Example Output
Predicted HDI Score	0.914
Category	Very High Human Development

Benefits

- Easy to understand
- Quick prediction
- Clear output
- Attractive interface



Milestone 5 — Deployment & Testing

5.1 Deployment Steps

16. Install Python dependencies.
17. Train and save the model as HDI.pkl.
18. Place the model inside the Flask project.
19. Run the Flask application.
20. Open the application in the browser.
21. Enter HDI parameters.
22. Verify prediction results.

5.2 Software Used

Software	Purpose
Python	Programming Language
Flask	Web Framework

Software	Purpose
Jupyter Notebook	Model Training
Visual Studio Code	Development
Git	Version Control
GitHub	Repository Management

5.3 Testing and Validation

The application was tested using different combinations of HDI input values.

Test Case	Expected Result	Status
Valid input	Predict HDI successfully	Pass
Invalid input	Validation error	Pass
Large GNI value	Prediction generated	Pass
Empty fields	Input validation triggered	Pass

➤ Performance

- Fast prediction
- Accurate output
- Responsive interface
- Stable Flask application
- Easy navigation

❖ Application Demonstration

Home Page (index.html)

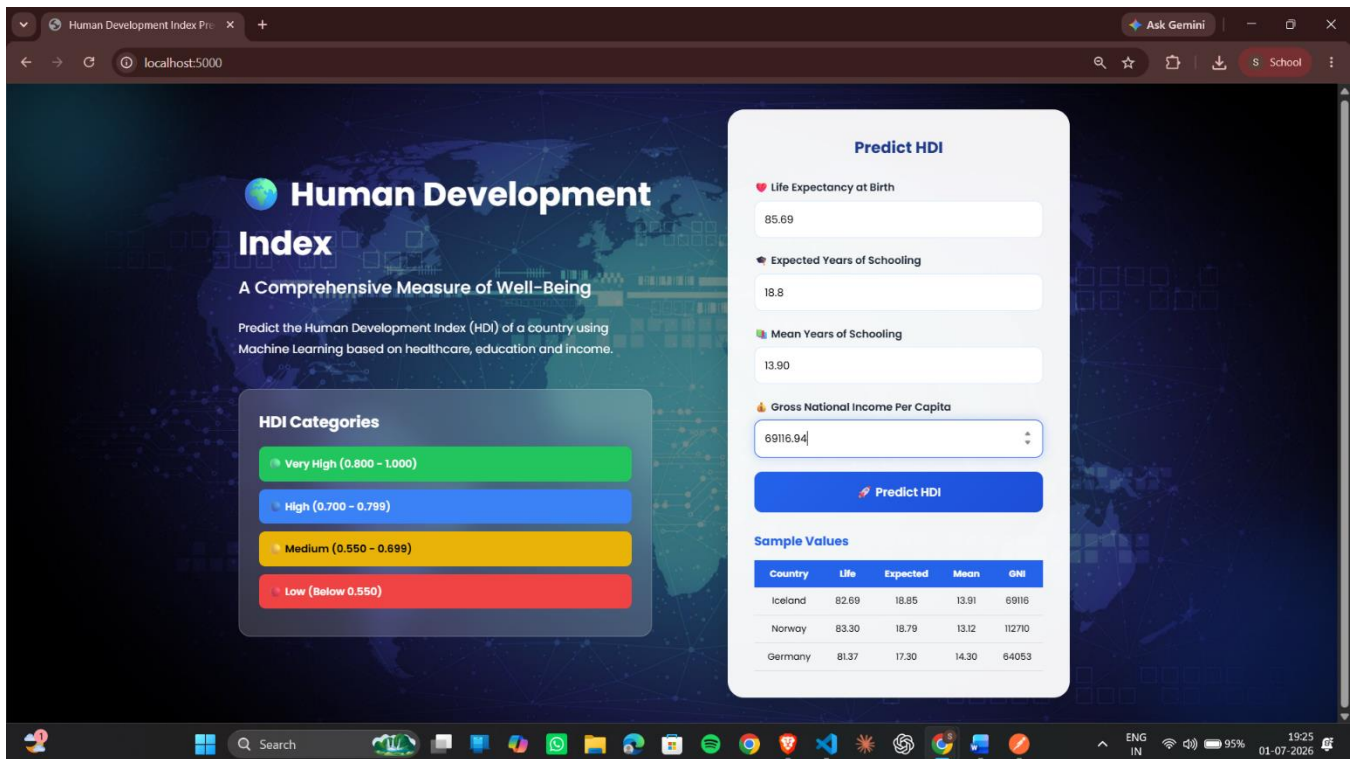
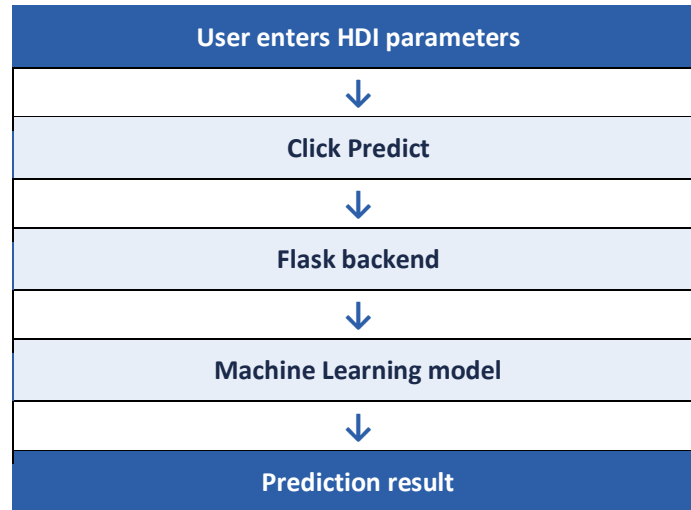
The Home Page is the main interface of the application where users enter the Human Development Index indicators required for prediction.

➤ Features

- Clean and responsive user interface
- User-friendly input form
- Input validation
- Predict button
- Easy navigation

➤ Input Parameters

- Life Expectancy at Birth
- Expected Years of Schooling
- Mean Years of Schooling
- Gross National Income (GNI) per Capita



Prediction Result Page (result.html)

The Result Page displays the predicted Human Development Index along with the corresponding development category.

Field	Example Output
Predicted HDI Score	0.914
Category	Very High Human Development

Features

- Displays prediction instantly
- Easy-to-read result format
- Attractive interface
- Reliable prediction output

Testing Summary

Test Case	Expected Result	Actual Result	Status
Valid input	HDI prediction generated	Prediction generated	Pass
Invalid input	Validation message	Validation displayed	Pass
Empty fields	Input required	Validation displayed	Pass
Large input values	Prediction generated	Prediction generated	Pass

Conclusion

The project A Comprehensive Measure of Well-Being successfully demonstrates the use of Machine Learning for predicting the Human Development Index based on key socio-economic indicators. The application combines a Linear Regression model with the Flask web framework to provide accurate and efficient HDI predictions through a simple web interface.

The system enables users to enter Life Expectancy at Birth, Expected Years of Schooling, Mean Years of Schooling, and Gross National Income (GNI) per Capita, and instantly obtain the predicted HDI score along with its corresponding development category.

This project illustrates the practical application of Machine Learning in analyzing human development indicators and provides a foundation for future improvements in development analytics.

Future Enhancements

- Implement advanced Machine Learning algorithms such as Random Forest and XGBoost.
- Support larger and updated HDI datasets.
- Add interactive charts and data visualizations.
- Enable comparative analysis between multiple countries.
- Deploy the application on cloud platforms such as Render or AWS.
- Develop a mobile-friendly version of the application.
- Add multilingual support.
- Integrate live development data from official sources.

References

23. United Nations Development Programme (UNDP) Human Development Reports.
24. Scikit-learn Documentation.
25. Flask Documentation.
26. Pandas Documentation.
27. NumPy Documentation.
28. Python Official Documentation.
29. Jupyter Notebook Documentation.
30. GitHub Documentation.

Project Summary

Item	Details
Project Name	A Comprehensive Measure of Well-Being
Machine Learning Algorithm	Linear Regression
Programming Language	Python
Framework	Flask
Frontend	HTML, CSS, JavaScript
Dataset	Human Development Index (HDI) Dataset
Model File	HDI.pkl
Output	Predicted HDI Score and Development Category

Project Deployment Link: <https://hdi-wellbeing-project.onrender.com>

GIT HUB REPO LINK: <https://github.com/AP24110010307/HDI-WellBeing-Project>